BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN

_____ o0o _____



BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: THUẬT TOÁN ỨNG DỤNG

ĐỀ TÀI: TỐI ƯU HÓA ĐA MỤC TIÊU QUY TRÌNH LẤY HÀNG TRONG KHO HÀNG

Đặng Anh Tuyền — **MSSV:** 23010912, **Lớp:** K17-KHMT(AI&KHDL)_1

Phạm Văn Sự — **MSSV:** 23010523, **Lớp:** K17-KHMT(AI&KHDL)_1

GVHD: TS. Trần Đình Tân

Hà Nội, Năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN

_____ o0o _____

Đặng Anh Tuyền **MSSV:** 23010912
Lớp: K17-KHMT(AI&KHDL)_1

Phạm Văn Sự **MSSV:** 23010523
Lớp: K17-KHMT(AI&KHDL)_1

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: THUẬT TOÁN ỨNG DỤNG

TỐI ƯU HÓA ĐỒNG THỜI GOM ĐƠN, ĐỊNH TUYẾN VÀ LẬP LỊCH TRONG KHO HÀNG

GVHD: TS. Trần Đình Tân

Hà Nội, Năm 2026

BẢNG PHÂN CHIA CÔNG VIỆC

STT	Họ và tên	Vai trò	Nội dung công việc	Tỷ lệ
1	Đặng Anh Tuyền	**Trưởng nhóm**	– Lập kế hoạch, phân công và theo dõi tiến độ chung. – Thiết kế mô hình toán học và cấu trúc dữ liệu đồ thị kho hàng. – Cài đặt thuật toán ALNS (Adaptive Large Neighborhood Search). – Xây dựng hệ thống sinh dữ liệu 5 kịch bản kho và benchmark đối chuẩn. – Phát triển giao diện Web Streamlit và mô phỏng Canvas. – Hoàn thiện báo cáo và slide thuyết trình.	**50%**
2	Phạm Văn Sự	Thành viên	– Tìm hiểu cơ sở lý thuyết gom đơn, định tuyến và lập lịch (JOBPRSP). – Cài đặt và cấu hình các thuật toán cơ sở B0, B1, B2, B3 và VNS/LNS. – Thực hiện thử nghiệm và thu thập số liệu đối chuẩn bộ dữ liệu Kris. – Vẽ biểu đồ so sánh, phân tích kết quả thực nghiệm và kiểm định nghiệm. – Rà soát nội dung, chính tả và chuẩn hóa báo cáo LaTeX. – Chuẩn bị slide và thuyết trình.	**50%**

Repository dự án: github.com/tuyenda2011/TTUD_v2

DANH MỤC TỪ VIẾT TẮT

Ký hiệu	Ý nghĩa
JOBPRSP	Bài toán tích hợp gom đơn, định tuyến và lập lịch nhân viên lấy hàng.
ALNS	Adaptive Large Neighborhood Search: tìm kiếm lân cận lớn thích ứng.
LNS	Large Neighborhood Search: tìm kiếm lân cận lớn.
VNS	Variable Neighborhood Search: tìm kiếm biến đổi lân cận.
SA	Simulated Annealing: luyện kim mô phỏng.
NN	Nearest Neighbor: láng giềng gần nhất.
EDD	Earliest Due Date: ưu tiên hạn hoàn thành sớm nhất.
FCFS	First Come, First Served: phục vụ theo thứ tự đến.
SKU	Stock Keeping Unit: mã phân biệt sản phẩm lưu kho.

Mục lục

DANH MỤC TỪ VIẾT TẮT **1**

1 TỔNG QUAN ĐỀ TÀI **7**
1.1 Bối cảnh và nhu cầu tối ưu quy trình lấy hàng trong kho 7
1.2 Bài toán tích hợp gom đơn, định tuyến và lập lịch 7
1.3 Các hướng giải quyết liên quan 8
1.4 Mục tiêu và phạm vi đề tài 9

2 MÔ HÌNH HÓA BÀI TOÁN **11**
2.1 Dữ liệu đầu vào và các giả định 11
2.2 Biểu diễn kho hàng bằng đồ thị 12
2.3 Biểu diễn phương án gom đơn, tuyến đi và lịch thực hiện 13
2.4 Các ràng buộc của phương án khả thi 13
2.5 Hàm mục tiêu và các chỉ tiêu đánh giá 14
2.6 Chuẩn hóa các thành phần và lựa chọn trọng số 15
2.7 Ví dụ tính toán minh họa 16

3 THIẾT KẾ VÀ PHÂN TÍCH CÁC THUẬT TOÁN **19**
3.1 Quy trình giải bài toán 19
3.2 Định tuyến, lập lịch và đánh giá nghiệm 20
3.2.1 Xây dựng tuyến bằng Nearest Neighbor 20
3.2.2 Cải thiện tuyến bằng 2-Opt 20
3.2.3 Phân công nhân viên và xây dựng lịch thực hiện 20
3.2.4 Tính giá trị mục tiêu và kiểm tra ràng buộc 21
3.3 First-Come, First-Served Batching 21
3.3.1 Ý tưởng và nguyên lý 21
3.3.2 Các bước thực hiện và giả mã 22
3.3.3 Ví dụ minh họa 22
3.3.4 Phân tích chi phí tính toán và đặc điểm thuật toán 22
3.4 Greedy Batching 23
3.4.1 Ý tưởng và nguyên lý 23
3.4.2 Các bước thực hiện và giả mã 23
3.4.3 Ví dụ minh họa 23
3.4.4 Phân tích chi phí tính toán và đặc điểm thuật toán 24
3.5 Large Neighborhood Search 24
3.5.1 Ý tưởng và nguyên lý 24

3.5.2 Các bước thực hiện và giả mã 26
3.5.3 Ví dụ minh họa 26
3.5.4 Phân tích chi phí tính toán và đặc điểm thuật toán 27
3.6 Adaptive Large Neighborhood Search 27
3.6.1 Ý tưởng và nguyên lý 27
3.6.2 Các bước thực hiện và giả mã 28
3.6.3 Ví dụ minh họa 29
3.6.4 Phân tích chi phí tính toán và đặc điểm thuật toán 29
3.7 Variable Neighborhood Search 30
3.7.1 Ý tưởng và nguyên lý 30
3.7.2 Các bước thực hiện và giả mã 31
3.7.3 Ví dụ minh họa 31
3.7.4 Phân tích chi phí tính toán và đặc điểm thuật toán 31
3.8 So sánh cơ chế và phân tích chi phí tính toán 32

4 THỰC NGHIỆM VÀ PHÂN TÍCH KẾT QUẢ 34
4.1 Thiết kế thực nghiệm 34
4.1.1 Câu hỏi và mục tiêu đánh giá 34
4.1.2 Các bộ dữ liệu và quy mô bài toán 34
4.1.3 Cấu hình thuật toán, ngân sách và số lần chạy 34
4.1.4 Môi trường thực nghiệm và phương pháp tổng hợp kết quả . . 34
4.2 Kết quả tổng hợp của năm thuật toán 34
4.2.1 Chất lượng nghiệm trên dữ liệu mô phỏng 34
4.2.2 Chất lượng nghiệm trên dữ liệu tham chiếu 34
4.2.3 Các chỉ tiêu vận hành và thời gian tính toán 34
4.3 Phân tích các trường hợp nổi bật 34
4.4 Phân tích quá trình hội tụ và độ ổn định 34
4.5 Khả năng mở rộng theo quy mô bài toán 34
4.6 Ảnh hưởng của ngân sách tìm kiếm 34
4.7 Ảnh hưởng của trọng số và đánh đổi giữa các mục tiêu 34
4.8 Giao diện minh họa và khả năng tái lập kết quả 34
4.9 Thảo luận tổng hợp và giới hạn thực nghiệm 34

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 35
5.1 Kết quả đạt được theo mục tiêu đề tài 35
5.2 Nhận xét về điều kiện phù hợp của từng thuật toán 35
5.3 Hạn chế và hướng phát triển 35

TÀI LIỆU THAM KHẢO **36**

Danh sách hình ảnh

2.1 Đồ thị lối đi của ví dụ bốn đơn. 16
3.1 Quy trình giải và kiểm chứng của năm phương pháp. 19

Danh sách bảng

1.1 Các hướng giải quyết liên quan và nội dung tham khảo. 9
2.1 Các ký hiệu sử dụng trong mô hình bài toán. 11
2.2 Đơn hàng trong ví dụ minh họa. 17
2.3 So sánh các chỉ tiêu của ba phương án trong ví dụ. 17
3.1 Giả mã B0 theo cài đặt của đề tài. 22
3.2 Giả mã B2 theo cài đặt của đề tài. 23
3.3 Giả mã LNS với lựa chọn toán tử đều. 26
3.4 Các lựa chọn chèn trong ví dụ tái thiết hai đơn. 26
3.5 Điểm thưởng cho toán tử trong cài đặt ALNS. 28
3.6 Giả mã ALNS với lựa chọn toán tử thích ứng. 28
3.7 Quan hệ giữa mức rung lắc và nhóm lân cận của VNS. 30
3.8 Giả mã VNS với cải thiện lân cận trên số mẫu giới hạn. 31
3.9 Đối chiếu các thành phần của năm phương pháp. 32
3.10 Các thành phần chi phí chính theo cài đặt. 33

1 TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh và nhu cầu tối ưu quy trình lấy hàng trong kho

Lấy hàng là quá trình thu gom sản phẩm tại các vị trí lưu trữ để đáp ứng yêu cầu của đơn hàng. Nhân viên di chuyển từ điểm tập kết đến các vị trí cần lấy, thu gom sản phẩm và quay về điểm tập kết khi kết thúc chuyến. Trong báo cáo, điểm tập kết được gọi là depot; một mã hàng phân biệt sản phẩm được gọi là Stock Keeping Unit (SKU). Một đơn có thể gồm nhiều mã hàng nằm ở các vị trí khác nhau, còn một chuyến có thể phục vụ nhiều đơn nếu tổng tải không vượt sức chứa xe đẩy.

Khi nhiều đơn cùng chờ xử lý, cách tổ chức lấy hàng ảnh hưởng đến cả quãng đường di chuyển và thời điểm hoàn thành. Phục vụ từng đơn riêng lẻ có thể làm nhân viên đi lặp lại các đoạn đường. Gom đơn cho phép chia sẻ lộ trình, nhưng cũng có thể kéo dài chuyến và khiến đơn có hạn gần phải chờ lâu hơn. Khi có nhiều nhân viên, việc phân công và sắp xếp chuyến còn quyết định mức cân bằng công việc giữa các nhân viên.

Tổng quan của de Koster và cộng sự [1] hệ thống hóa các quyết định về thiết kế và điều hành hoạt động lấy hàng, trong đó có định tuyến và gom đơn. Với mô hình được xét trong đề tài, yêu cầu đặt ra là xây dựng kế hoạch phục vụ đủ đơn, tuân thủ sức chứa và các lối đi của kho, đồng thời cân đối quãng đường, thời gian hoàn tất và trễ hạn. Đây là cơ sở để lựa chọn một bài toán tối ưu có sự kết hợp giữa nhiều quyết định vận hành.

1.2 Bài toán tích hợp gom đơn, định tuyến và lập lịch

Đề tài xét bài toán tích hợp ba quyết định: gom đơn xác định các đơn được phục vụ trong cùng một chuyến; định tuyến xác định thứ tự ghé các vị trí lấy hàng và đường đi giữa chúng; lập lịch xác định nhân viên phụ trách cùng thứ tự thực hiện các chuyến của mỗi người. Một nghiệm hoàn chỉnh phải thể hiện được cả ba quyết định này.

Các quyết định có quan hệ chặt chẽ. Thêm một đơn vào chuyến làm thay đổi tập vị trí cần ghé và có thể làm tăng thời lượng chuyến. Các chuyến tiếp theo của cùng nhân viên khi đó phải bắt đầu muộn hơn, ảnh hưởng đến thời điểm hoàn thành những đơn khác. Ngược lại, thay đổi nhân viên phụ trách hoặc thứ tự chuyến có thể giảm trễ mà không cần thay đổi cách gom đơn.

Chẳng hạn, hai đơn có sản phẩm gần nhau nhưng hạn hoàn thành khác nhau có thể được gom để chia sẻ lộ trình. Tuy nhiên, trong mô hình của đề tài, đơn chỉ hoàn thành khi chuyến chứa nó quay về depot; vì vậy đơn gấp có thể phải chờ việc lấy hàng của đơn còn lại. Tách đơn gấp thành chuyến riêng có thể giảm trễ nhưng làm tăng quãng

đường. Ví dụ này cho thấy một thay đổi có lợi cho một chỉ tiêu chưa chắc có lợi cho các chỉ tiêu còn lại.

Đề tài sử dụng hạn hoàn thành mềm: một đơn được phép hoàn thành sau hạn, nhưng phần thời gian vượt hạn được tính vào chi phí. Chất lượng nghiệm được đánh giá bằng tổng có trọng số của ba thành phần đã chuẩn hóa: tổng quãng đường, thời gian hoàn tất toàn bộ công việc (makespan) và tổng độ trễ. Số đơn trễ được theo dõi như một chỉ tiêu bổ sung. Khi phân tích thực nghiệm, cần xem cả mục tiêu tổng hợp lẫn từng thành phần để nhận diện các đánh đổi.

1.3 Các hướng giải quyết liên quan

Các nghiên cứu về lấy hàng có thể tập trung vào một quyết định riêng hoặc xét nhiều quyết định đồng thời. Gademann và van de Velde [7] nghiên cứu gom đơn nhằm giảm tổng thời gian di chuyển trong kho có các lối song song. Ratliff và Rosenthal [8] xây dựng phương pháp định tuyến cho một trường hợp kho chữ nhật với lối nối ở hai đầu dãy. Hai công trình cung cấp cơ sở tham khảo cho việc tổ chức nhóm đơn và khai thác cấu trúc đường đi.

Ở hướng tích hợp, Briant và cộng sự [2] xét bài toán gom đơn, định tuyến và sắp thứ tự có hạn hoàn thành, sử dụng mô hình quy hoạch tuyến tính và phương pháp sinh cột để xây dựng cận dưới và cận trên. Đề tài tham khảo cách tiếp cận tích hợp, nhưng sử dụng mô hình có hạn mềm và mục tiêu gồm ba thành phần. Vì khác biệt về mô hình và tiêu chí đánh giá, kết quả được đối chiếu giữa các phương pháp cài đặt trong đề tài trên cùng dữ liệu và cùng bộ đánh giá.

Về tìm kiếm nghiệm, Shaw [3] trình bày Large Neighborhood Search (LNS), thay đổi một phần nghiệm bằng cách loại bỏ và chèn lại các phần tử. Ropke và Pisinger [4] phát triển Adaptive Large Neighborhood Search (ALNS), trong đó tần suất sử dụng các toán tử phụ thuộc vào hiệu quả đã quan sát. Mladenović và Hansen [5] đề xuất Variable Neighborhood Search (VNS), dựa trên việc thay đổi có hệ thống cấu trúc lân cận. Các khung này là cơ sở để xây dựng những chiến lược cải thiện nghiệm khác nhau cho bài toán kho hàng.

Tài liệu	Hướng tiếp cận	Nội dung tham khảo
de Koster và cộng sự [1]	Tổng quan hoạt động lấy hàng.	Các quyết định về gom đơn và định tuyến.
Gademann–van de Velde [7]	Gom đơn theo thời gian di chuyển.	Quan hệ giữa nhóm đơn và chi phí lấy hàng.
Ratliff–Rosenthal [8]	Định tuyến trong kho chữ nhật.	Khai thác cấu trúc lối đi.
Briant và cộng sự [2]	Tối ưu tích hợp có hạn hoàn thành.	Mô hình tích hợp và cách xây dựng cận.
Shaw [3]	LNS.	Loại bỏ và tái chèn một phần nghiệm.
Ropke–Pisinger [4]	ALNS.	Lựa chọn toán tử theo hiệu quả quan sát.
Mladenović–Hansen [5]	VNS.	Thay đổi cấu trúc lân cận.

Bảng 1.1: Các hướng giải quyết liên quan và nội dung tham khảo.

Trên cơ sở các hướng trong Bảng 1.1, đề tài cài đặt năm phương pháp: First-Come, First-Served Batching (B0), Greedy Batching (B2), LNS, ALNS và VNS. B0 là mốc kiến tạo đơn giản, gom đơn theo thứ tự đầu vào; B2 sử dụng tiêu chí tham lam để xây dựng nghiệm và làm điểm xuất phát cho ba phương pháp tìm kiếm. So sánh LNS với ALNS nhằm khảo sát lựa chọn toán tử đều và lựa chọn thích ứng trên cùng bộ toán tử. VNS bổ sung một chiến lược đối chứng dựa trên thay đổi cấu trúc lân cận. Cách lựa chọn này cho phép đánh giá từ nghiệm cơ sở đến các cơ chế cải thiện nghiệm, với cùng mô hình và hàm mục tiêu.

Trong cài đặt, B0 dựng tuyến bằng Nearest Neighbor, còn B2 kết hợp Nearest Neighbor với 2-Opt. Vì vậy, so sánh B2 với B0 phản ánh sự khác biệt của cả cách gom đơn và cách định tuyến; không quy toàn bộ chênh lệch chất lượng nghiệm cho riêng bước gom đơn tham lam. Các đối chiếu với B2 cho biết lợi ích của việc tiếp tục tìm kiếm trong ngân sách đã xét, nhưng không bảo đảm một phương pháp luôn tốt hơn trên mọi đầu vào.

1.4 Mục tiêu và phạm vi đề tài

Mục tiêu của đề tài là xây dựng và phân tích các thuật toán giải bài toán tích hợp gom đơn, định tuyến và lập lịch, đồng thời đánh giá chất lượng nghiệm và chi phí tính toán của năm phương pháp. Phần thuật toán trình bày nguyên lý, giả mã, ví dụ minh họa và phân tích chi phí của cài đặt. Phần thực nghiệm đối chiếu kết quả trên các nhóm dữ liệu, liên hệ những khác biệt quan sát được với cơ chế tìm kiếm và điều kiện chạy.

Trong phần thực nghiệm, thời gian tính toán là thời gian chương trình chạy để xây dựng và kiểm chứng kế hoạch. Đại lượng này được phân biệt với makespan, là thời

gian hoàn tất công việc theo lịch thực hiện trong mô hình kho. Một thuật toán chạy nhanh trên máy tính chưa chắc tạo ra kế hoạch có makespan thấp, và ngược lại.

Các sản phẩm chính gồm bộ giải cài đặt năm phương pháp trên cùng mô hình và bộ đánh giá; cơ chế kiểm chứng tính hợp lệ của nghiệm cùng các chỉ tiêu được xuất ra; và bộ thực nghiệm lưu cấu hình, đầu vào, nghiệm và số liệu để tổng hợp kết quả so sánh. Giao diện mô phỏng hỗ trợ quan sát tuyến đi và lịch thực hiện. Đóng góp của đề tài tập trung vào việc áp dụng, cài đặt và kiểm chứng các phương pháp trong phạm vi bài toán đã xác định, không khẳng định một khung thuật toán mới.

Các câu hỏi đánh giá chính gồm:

1. Chất lượng nghiệm của B2 khác B0 như thế nào?
2. Tiếp tục tìm kiếm từ nghiệm B2 làm thay đổi chất lượng nghiệm và thời gian tính toán như thế nào?
3. Với cùng bộ toán tử và ngân sách, lựa chọn thích ứng của ALNS cho kết quả khác LNS như thế nào?
4. Chiến lược thay đổi lân cận của VNS cho chất lượng nghiệm, độ ổn định và thời gian tính toán như thế nào so với LNS và ALNS?
5. Quy mô bài toán và ngân sách tìm kiếm ảnh hưởng thế nào đến chất lượng nghiệm và thứ hạng các phương pháp?
6. Thay đổi trọng số mục tiêu tạo ra những đánh đổi nào giữa quãng đường, thời gian hoàn tất và độ trễ?

Phạm vi nghiên cứu là kho tĩnh với số nhân viên cho trước, nhân viên đồng nhất về tốc độ và thời gian thao tác, cùng xe đẩy có sức chứa hữu hạn. Toàn bộ đơn được biết và sẵn sàng tại thời điểm bắt đầu; mỗi đơn được phục vụ trọn vẹn trong một chuyến và hoàn thành khi chuyến quay về depot. Mô hình chưa xét đơn phát sinh trong quá trình vận hành, ca nghỉ hoặc xung đột di chuyển giữa nhân viên.

Thực nghiệm sử dụng các bản đồ kho tổng hợp, nhóm dữ liệu thay đổi quy mô và bộ dữ liệu tham chiếu Kris Small. Kết quả được tổng hợp riêng theo nhóm dữ liệu; các trường hợp nổi bật được lựa chọn để phân tích cùng với bảng kết quả đầy đủ. Việc đánh giá bao gồm kiểm tra tính hợp lệ của nghiệm, chất lượng mục tiêu, các thành phần vận hành, hội tụ và độ ổn định qua các lần chạy. Các kết luận được giới hạn trong mô hình, dữ liệu và ngân sách đã khảo sát; kết quả trên dữ liệu mô phỏng chưa đủ để khẳng định hiệu quả trong mọi kho thực tế hoặc bảo đảm tối ưu toàn cục.

2 MÔ HÌNH HÓA BÀI TOÁN

2.1 Dữ liệu đầu vào và các giả định

Một đầu vào xác định kho hàng, tập đơn cần phục vụ và các thông số vận hành. Gọi O là tập đơn, K là tập sản phẩm và $P = \{1, \ldots, m\}$ là tập nhân viên, với $n = |O| > 0$. Đơn $i \in O$ yêu cầu số lượng q_{is} của sản phẩm s; quy ước $q_{is} = 0$ nếu đơn không yêu cầu sản phẩm đó. Mỗi sản phẩm có vị trí lấy hàng ℓ_s, tải của một đơn vị $a_s > 0$ và thời gian lấy một đơn vị $h_s \geq 0$. Tải của đơn được tính bởi

$$w_i = \sum_{s \in K} a_s q_{is}. \tag{2.1}$$

Đơn có hạn hoàn thành $d_i \geq 0$ và một thứ tự đầu vào. Thứ tự này phục vụ việc gom đơn cơ sở, không phải thời điểm đơn phát sinh.

Thông số vận hành gồm sức chứa xe $Q > 0$, vận tốc di chuyển $v > 0$, thời gian chuẩn bị mỗi chuyến $b \geq 0$ và thời gian thao tác cố định tại mỗi vị trí có yêu cầu lấy hàng $h_{\text{loc}} \geq 0$. Sức chứa và tải sản phẩm dùng cùng một đơn vị tải quy ước, không mặc định là khối lượng. Dữ liệu mô phỏng dùng mét và phút; dữ liệu Kris giữ quy ước số học của nguồn. Trong từng đầu vào, khoảng cách, vận tốc và thời gian phải nhất quán. Việc quy đổi đơn vị hiển thị không thay thế việc đánh giá nghiệm theo dữ liệu nguồn.

Ký hiệu	Ý nghĩa
O, K, P	Tập đơn, tập sản phẩm và tập nhân viên.
n, m	Số đơn và số nhân viên.
q_{is}, a_s, h_s	Số lượng yêu cầu, tải và thời gian lấy một đơn vị sản phẩm.
ℓ_s, w_i, d_i	Vị trí sản phẩm, tải đơn và hạn hoàn thành.
Q, v, b, h_{loc}	Sức chứa, vận tốc, thời gian chuẩn bị và thao tác tại vị trí.
$B_{p,j}, J_p$	Tập đơn trong chuyến thứ j và số chuyến của nhân viên p.
$U(B), D(B), \tau(B)$	Tập vị trí cần lấy hàng, quãng đường và thời lượng chuyến.
$S_{p,j}, C_{p,j}$	Thời điểm bắt đầu và kết thúc chuyến.
C_i, T_i	Thời điểm hoàn thành và độ trễ của đơn.
$D, C_{\max}, T, L$	Tổng quãng đường, makespan, tổng độ trễ và số đơn trễ.
t_{run}	Thời gian máy tính chạy để xây dựng và kiểm chứng kế hoạch.
$\alpha_D, \alpha_C, \alpha_T$	Trọng số của ba thành phần trong hàm mục tiêu.

Bảng 2.1: Các ký hiệu sử dụng trong mô hình bài toán.

Mô hình sử dụng các giả định sau:

1. Toàn bộ đơn được biết và sẵn sàng tại thời điểm 0; không bổ sung đơn trong khi thực hiện kế hoạch.

2. Nhân viên đồng nhất về vận tốc và thời gian thao tác. Mỗi người dùng xe có cùng sức chứa và thực hiện tối đa một chuyến tại một thời điểm.
3. Mỗi đơn được phục vụ trọn vẹn trong đúng một chuyến. Đầu vào phải thỏa $w_i \leq Q$, nên một đơn riêng lẻ không cần bị tách để đáp ứng sức chứa.
4. Mỗi chuyến bắt đầu và kết thúc tại depot. Đơn hoàn thành khi chuyến chứa nó quay về depot.
5. Hạn hoàn thành là mềm: trễ hạn làm tăng chi phí nhưng không làm nghiệm mất tính khả thi.
6. Không xét tắc nghẽn, xung đột di chuyển, thiếu hàng, ca nghỉ hoặc sự thay đổi vận tốc theo tải.

Các giả định xác định phạm vi áp dụng của kết quả. Một nghiệm hợp lệ trong mô hình chưa đủ để bảo đảm đáp ứng mọi điều kiện vận hành của một kho thực tế.

2.2 Biểu diễn kho hàng bằng đồ thị

Kho được biểu diễn bằng đồ thị vô hướng liên thông $G = (V, E)$ có trọng số cạnh không âm. Tập V gồm depot, vị trí sản phẩm và các điểm trung chuyển; cạnh $(u, v) \in E$ biểu diễn một đoạn lối đi được phép sử dụng. Gọi c_{uv} là chiều dài của đoạn đó và $d_0 \in V$ là depot. Tọa độ của đỉnh dùng để thể hiện bố trí kho; quãng đường được tính từ các cạnh, không dùng khoảng cách đường thẳng xuyên qua kệ.

Khoảng cách giữa hai vị trí là độ dài đường đi ngắn nhất trên đồ thị:

$$\delta(u, v) = \min_{\pi : u \rightsquigarrow v} \sum_{(x,y) \in \pi} c_{xy}. \tag{2.2}$$

Với trọng số không âm, chương trình dùng Dijkstra để tính khoảng cách và lưu thông tin truy vết đường đi. Do đồ thị liên thông, mọi vị trí cần phục vụ đều có thể tiếp cận từ depot.

Với chuyến chứa tập đơn B, tập vị trí có yêu cầu lấy hàng là

$$U(B) = \bigcup_{i \in B} \{\ell_s : q_{is} > 0\}. \tag{2.3}$$

Nhiều sản phẩm hoặc nhiều đơn có thể dùng chung một vị trí; vị trí đó chỉ xuất hiện một lần trong $U(B)$. Nếu thứ tự điểm dừng là $(u_0, u_1, \ldots, u_r, u_{r+1})$, trong đó $u_0 = u_{r+1} = d_0$ và các điểm lấy hàng ngoài depot nằm giữa hai đầu, quãng đường chuyến được tính bởi

$$D(B) = \sum_{t=0}^{r} \delta(u_t, u_{t+1}). \tag{2.4}$$

Từng đoạn nối điểm dừng sau đó được mở rộng thành đường đi trên các cạnh của kho. Vì vậy, dãy điểm dừng phục vụ và đường đi vật lý là hai cách biểu diễn khác nhau: đường đi vật lý có thể qua một đỉnh nhiều lần dù vị trí lấy hàng chỉ được liệt kê một lần trong dãy điểm dừng.

2.3 Biểu diễn phương án gom đơn, tuyến đi và lịch thực hiện

Ở mức lập kế hoạch, một phương án cần xác định các đơn thuộc từng chuyến, tuyến phục vụ của chuyến, nhân viên phụ trách và thứ tự thực hiện. Trong cài đặt, cấu trúc nghiệm chính là danh sách chuyến có thứ tự của từng nhân viên:

$$X = (\sigma_1, \ldots, \sigma_m), \qquad \sigma_p = (B_{p,1}, \ldots, B_{p,J_p}). \tag{2.5}$$

Một nhân viên không được giao việc có $J_p = 0$. Tập đơn của chuyến quyết định $U(B)$; thủ tục định tuyến dựng tuyến trên đồ thị, rồi bộ đánh giá tính thời lượng và lịch theo thứ tự chuyến.

Như vậy, cấu trúc nghiệm trong chương trình lưu trực tiếp phân hoạch đơn và thứ tự chuyến, còn tuyến và thời điểm được suy ra bằng bộ giải mã. Đây là cách áp dụng heuristic cho bài toán tích hợp; chương trình không tìm kiếm độc lập trên toàn bộ các tuyến và các thời điểm bắt đầu có thể có. Quy tắc định tuyến cụ thể của từng thuật toán được trình bày ở Chương 3.

Thay đổi một đơn giữa hai chuyến có thể làm thay đổi cả tuyến và thời lượng. Đổi thứ tự hai chuyến hoặc chuyển chuyến sang nhân viên khác có thể làm thay đổi nhiều thời điểm hoàn thành phía sau. Vì vậy, sau khi thay đổi nghiệm, cần tính lại các chỉ tiêu liên quan trước khi so sánh chất lượng.

2.4 Các ràng buộc của phương án khả thi

Gọi $\mathcal{F}$ là tập các cấu trúc nghiệm hoàn chỉnh thỏa các điều kiện dưới đây. Trước hết, mọi đơn phải được phục vụ đúng một lần:

$$\bigcup_{p=1}^{m} \bigcup_{j=1}^{J_p} B_{p,j} = O, \tag{2.6}$$

$$B_{p,j} \cap B_{q,k} = \varnothing \quad \text{khi } (p,j) \neq (q,k). \tag{2.7}$$

Mỗi chuyến phải chứa ít nhất một đơn và tuân thủ sức chứa:

$$B_{p,j} \neq \varnothing, \qquad \sum_{i \in B_{p,j}} w_i \leq Q. \tag{2.8}$$

Nghiệm chỉ sử dụng các đơn có trong đầu vào. Trạng thái trung gian tạm thiếu đơn trong quá trình tìm kiếm không được xem là một nghiệm đầu ra hoàn chỉnh.

Tuyến của mỗi chuyến phải bắt đầu và kết thúc tại depot, phục vụ đủ $U(B)$ và chỉ di chuyển trên các cạnh hợp lệ. Nhân viên thực hiện các chuyến tuần tự; chuyến sau không bắt đầu trước khi chuyến trước kết thúc:

$$S_{p,1} \geq 0, \qquad S_{p,j+1} \geq C_{p,j}. \tag{2.9}$$

Trong cài đặt, chuyến đầu bắt đầu tại 0, chuyến sau bắt đầu ngay khi chuyến trước kết thúc. Với toàn bộ đơn sẵn sàng tại 0 và không có ràng buộc giờ mở cửa hoặc thời điểm sớm nhất, loại bỏ thời gian chờ trong một thứ tự chuyến cố định không làm tăng quãng đường, makespan hoặc độ trễ.

Không đặt ràng buộc $C_i \leq d_i$ vì mô hình cho phép trễ hạn. Cần phân biệt kiểm chứng khả thi với đánh giá chất lượng: nghiệm đủ đơn và đúng sức chứa vẫn có thể có độ trễ lớn hoặc giá trị mục tiêu cao. Chương trình kiểm tra đầu vào, cấu trúc nghiệm và đối chiếu tuyến, thời gian, chỉ tiêu trong kết quả xuất ra; các phép so sánh số thực có sử dụng dung sai.

2.5 Hàm mục tiêu và các chỉ tiêu đánh giá

Thời lượng chuyến gồm chuẩn bị, di chuyển, thao tác tại các vị trí cần lấy hàng và lấy từng đơn vị sản phẩm:

$$\tau(B) = b + \frac{D(B)}{v} + h_{\text{loc}}|U(B)| + \sum_{i \in B} \sum_{s \in K} h_s q_{is}. \tag{2.10}$$

Thời gian thao tác tại vị trí được tính một lần cho mỗi vị trí có yêu cầu trong chuyến, kể cả khi nhiều đơn dùng chung vị trí đó. Thời gian lấy sản phẩm vẫn tính theo tổng số lượng. Việc đường đi vật lý qua một vị trí nhiều lần không làm tự động tăng số lần tính thao tác tại vị trí trong mô hình.

Lịch được tính liên tiếp theo từng nhân viên:

$$C_{p,0} = 0, \tag{2.11}$$

$$S_{p,j} = C_{p,j-1}, \qquad C_{p,j} = S_{p,j} + \tau(B_{p,j}). \tag{2.12}$$

Mọi đơn thuộc một chuyến có cùng thời điểm hoàn thành, bằng thời điểm chuyến quay về depot:

$$C_i = C_{p,j} \quad \text{với } i \in B_{p,j}, \qquad T_i = \max(0, C_i - d_i). \tag{2.13}$$

Ba thành phần chính là tổng quãng đường, thời gian hoàn tất toàn bộ công việc và tổng độ trễ:

$$D(X) = \sum_{p=1}^{m} \sum_{j=1}^{J_p} D(B_{p,j}), \tag{2.14}$$

$$C_{\max}(X) = \max_{p \in P} C_{p,J_p}, \tag{2.15}$$

$$T(X) = \sum_{i \in O} T_i. \tag{2.16}$$

Quy ước $C_{p,J_p} = 0$ cho nhân viên không có chuyến. Makespan là thời điểm người cuối cùng kết thúc, không phải tổng thời gian làm việc của các nhân viên. Tổng độ trễ cộng phần vượt hạn của từng đơn, nên một chuyến có nhiều đơn trễ có thể đóng góp nhiều lần vào T.

Các đại lượng $S_{p,j}$, $C_{p,j}$ và $C_{\max}$ thuộc lịch vận hành, được tính từ tuyến và thời gian thao tác; với dữ liệu mô phỏng, chúng có đơn vị phút. Ngược lại, thời gian tính toán t_{run} được đo bằng đồng hồ máy tính theo giây, bao gồm tiền xử lý, khởi tạo, tìm kiếm và kiểm chứng của lượt giải. Đây là chỉ tiêu đánh giá chi phí tính toán, không phải thời lượng chuyến và không nằm trong hàm mục tiêu F. Khi phân tích kết quả, thời gian tính toán và makespan được báo cáo riêng.

Chương trình ghi thêm số đơn trễ và tỷ lệ đơn đúng hạn:

$$L(X) = \sum_{i \in O} \mathbf{1}\{T_i > \varepsilon\}, \qquad R_{\text{on}}(X) = 1 - \frac{L(X)}{n}, \qquad \varepsilon = 10^{-9}. \tag{2.17}$$

L là số đếm, còn T là tổng thời gian; hai chỉ tiêu phản ánh các khía cạnh khác nhau. Dung sai ε được dùng trong cài đặt để nhận diện trễ khi tính bằng số thực. Các công thức thời lượng, lịch và chỉ tiêu trong mục này tương ứng với bộ đánh giá của chương trình.

2.6 Chuẩn hóa các thành phần và lựa chọn trọng số

Để kết hợp các đại lượng có đơn vị và thang giá trị khác nhau, đề tài dùng nghiệm First-Come, First-Served Batching (B0) làm mốc trên từng đầu vào. Gọi D_0 và C_0 là tổng quãng đường và makespan của nghiệm này, tính với quy tắc định tuyến cơ sở. Các thang tham chiếu trong cài đặt là

$$D_{\text{ref}} = \max(D_0, 1), \qquad C_{\text{ref}} = \max(C_0, 1), \qquad T_{\text{ref}} = nC_{\text{ref}}. \tag{2.18}$$

Giá trị 1 là sàn số học trong quy ước đơn vị đầu vào để tránh mẫu số bằng không. Thang độ trễ không lấy từ $T(B0)$: ngay cả khi B0 không có đơn trễ, nC_{ref} vẫn dương.

Các thang này được tính một lần từ B0 và giữ cố định khi đánh giá các phương pháp, seed tìm kiếm và cấu hình trọng số trên cùng đầu vào.

Hàm mục tiêu cần tối thiểu hóa là

$$\min_{X \in \mathcal{F}} F(X), \qquad F(X) = \alpha_D \frac{D(X)}{D_{\text{ref}}} + \alpha_C \frac{C_{\max}(X)}{C_{\text{ref}}} + \alpha_T \frac{T(X)}{T_{\text{ref}}}, \tag{2.19}$$

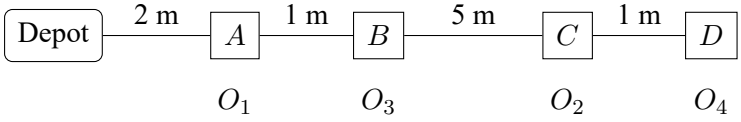
với $\alpha_D, \alpha_C, \alpha_T > 0$ và $\alpha_D + \alpha_C + \alpha_T = 1$. Tuyến và lịch trong $F(X)$ được giải mã theo quy tắc của phương pháp đang xét. Biểu thức tối thiểu hóa xác định mục tiêu của bộ giải, không khẳng định các heuristic tìm được nghiệm tối ưu của tập khả thi.

Cấu hình tham chiếu dùng $(\alpha_D, \alpha_C, \alpha_T) = (1/3, 1/3, 1/3)$. Trọng số bằng nhau sau chuẩn hóa là quy ước đối chiếu, chưa được hiệu chuẩn từ chi phí kinh tế hoặc mức phục vụ của một kho thực tế. Thành phần $T/(nC_{\text{ref}})$ chính là độ trễ trung bình mỗi đơn chia cho makespan tham chiếu. Số đơn trễ L không nằm trực tiếp trong F.

Giá trị F thấp hơn biểu thị phương án tốt hơn theo trọng số đã chọn. Tuy nhiên, giảm F không bảo đảm cả ba thành phần đều giảm, và tăng trọng số độ trễ không đồng nghĩa trực tiếp với tối thiểu hóa số đơn trễ. Chỉ đối chiếu điểm F khi dùng cùng đầu vào, cùng thang tham chiếu và cùng trọng số; giữa các cấu hình trọng số, cần xem thêm D, $C_{\max}$ và T. Các thang tham chiếu được xác định riêng cho từng đầu vào, nên không diễn giải chênh lệch điểm giữa hai bài khác nhau như chênh lệch chi phí vận hành tuyệt đối.

2.7 Ví dụ tính toán minh họa

Xét một kho nhỏ có bốn đơn và hai nhân viên. Depot cùng bốn vị trí sản phẩm nằm trên một lối đi như Hình 2.1. Mỗi cạnh có thể đi theo hai chiều; số ghi trên cạnh là chiều dài thực, còn sơ đồ không vẽ theo tỷ lệ.



Hình 2.1: Đồ thị lối đi của ví dụ bốn đơn.

Mỗi đơn yêu cầu một đơn vị sản phẩm, có tải 1 và thời gian lấy 1 phút. Sức chứa xe $Q = 2$, vận tốc $v = 2$ mét/phút, thời gian chuẩn bị $b = 1$ phút và thời gian thao tác tại vị trí $h_{\text{loc}} = 0$. Thứ tự đầu vào là O_1, O_2, O_3, O_4; hạn và vị trí của từng đơn được cho trong Bảng 2.2.

Đơn	Vị trí sản phẩm	Tải	Hạn, phút
O_1	A	1	7
O_2	C	1	12
O_3	B	1	7
O_4	D	1	12

Bảng 2.2: Đơn hàng trong ví dụ minh họa.

B0 gom theo thứ tự thành hai chuyến $\{O_1, O_2\}$ và $\{O_3, O_4\}$, giao cho hai nhân viên và cùng bắt đầu tại 0. Chuyến thứ nhất đi đến C rồi quay về, với điểm dừng $d_0 \to A \to C \to d_0$ và quãng đường $2 + 6 + 8 = 16$ mét. Theo (2.10), thời lượng là $1 + 16/2 + 2 = 11$ phút. Chuyến thứ hai có quãng đường $3 + 6 + 9 = 18$ mét và thời lượng $1 + 18/2 + 2 = 12$ phút.

Vì đơn hoàn thành khi chuyến quay về, O_1 trễ $11 - 7 = 4$ phút và O_3 trễ $12 - 7 = 5$ phút; hai đơn còn lại đúng hạn. Nghiệm B0 vẫn khả thi vì mỗi chuyến có tải 2 và đủ bốn đơn. Các chỉ tiêu là $D = 34$, $C_{\max} = 12$, $T = 9$, $L = 2$ và $R_{\text{on}} = 50\%$. Thang chuẩn hóa là $D_{\text{ref}} = 34$, $C_{\text{ref}} = 12$ và $T_{\text{ref}} = 4 \times 12 = 48$.

Với trọng số bằng nhau, điểm của B0 được tính bởi

$$F(B0) = \frac{1}{3}\left(\frac{34}{34} + \frac{12}{12} + \frac{9}{48}\right) = \frac{35}{48} \approx 0{,}729167. \tag{2.20}$$

Trên cùng đầu vào, Greedy Batching (B2) tạo hai chuyến $\{O_1, O_3\}$ và $\{O_2, O_4\}$ trong cài đặt hiện tại. Chuyến gần depot có quãng đường $2 + 1 + 3 = 6$ mét và thời lượng $1 + 6/2 + 2 = 6$ phút. Chuyến xa có quãng đường $8 + 1 + 9 = 18$ mét và thời lượng 12 phút. Mỗi nhân viên nhận một chuyến; cả bốn đơn đều đúng hạn.

Để minh họa riêng ảnh hưởng của lập lịch, xét phương án X giữ nguyên hai chuyến và tuyến của B2 nhưng giao cả hai cho nhân viên thứ nhất, thực hiện chuyến gần trước. Nhân viên thứ hai không nhận chuyến. Chuyến đầu bắt đầu tại 0 và kết thúc tại 6; chuyến sau bắt đầu tại 6 và kết thúc tại 18 phút. Hai đơn gần depot vẫn đúng hạn, còn O_2, O_4 cùng trễ $18 - 12 = 6$ phút. Vì vậy $D = 24$, $C_{\max} = 18$, $T = 12$ và $L = 2$. Đây là phương án phân công khác dùng để minh họa, không phải đầu ra của B2.

Phương án	D, m	$C_{\max}$, phút	T, phút	L	F
B0	34	12	9	2	0,729167
B2	24	12	0	0	0,568627
X, một nhân viên	24	18	12	2	0,818627

Bảng 2.3: So sánh các chỉ tiêu của ba phương án trong ví dụ.

Dùng cùng thang chuẩn hóa của B0, điểm của B2 là

$$F(B2) = \frac{1}{3}\left(\frac{24}{34} + \frac{12}{12} + \frac{0}{48}\right) = \frac{29}{51} \approx 0{,}568627. \tag{2.21}$$

Theo Bảng 2.3, B2 giảm 10 mét, giữ nguyên makespan và loại độ trễ trong ví dụ. Mức giảm mục tiêu so với B0 là

$$100\frac{F(B0) - F(B2)}{F(B0)} \approx 22{,}02\%. \tag{2.22}$$

Dùng cùng thang chuẩn hóa của B0, điểm của phương án X là

$$F(X) = \frac{1}{3}\left(\frac{24}{34} + \frac{18}{12} + \frac{12}{48}\right) = \frac{167}{204} \approx 0{,}818627. \tag{2.23}$$

So với B2, phương án X giữ nguyên quãng đường nhưng có makespan và độ trễ lớn hơn do dồn hai chuyến vào một nhân viên. Ví dụ này làm rõ rằng đánh giá cách gom đơn và tuyến đi cần đi cùng đánh giá phân công và lịch thực hiện.

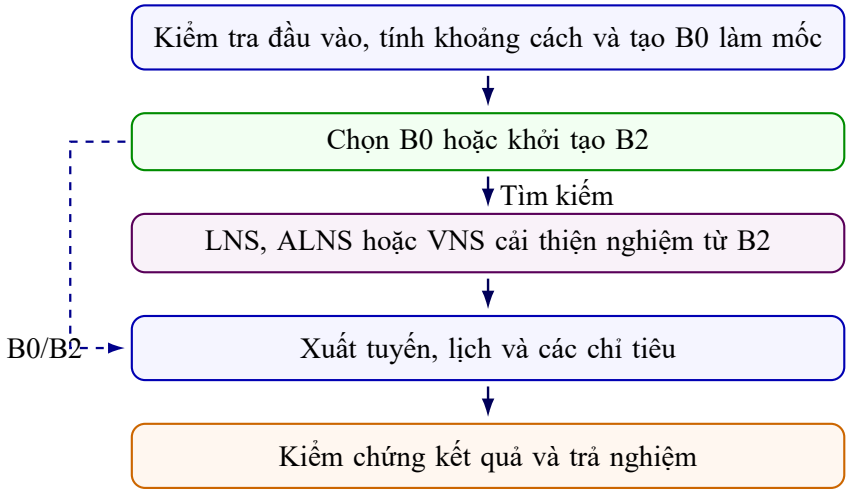
Ví dụ cho thấy cách tính các nghiệm khả thi và các thành phần mục tiêu. Cả ba phương án đã được đánh giá và kiểm chứng chỉ tiêu bằng chương trình, nhưng không được cộng vào benchmark hoặc dùng làm bằng chứng về ưu thế trên toàn bộ dữ liệu. Ví dụ cũng không chứng minh B2 đạt tối ưu toàn cục; cơ chế tạo hai cách gom được giải thích tiếp ở Chương 3.

3 THIẾT KẾ VÀ PHÂN TÍCH CÁC THUẬT TOÁN

3.1 Quy trình giải bài toán

Chương này trình bày cách năm phương pháp được áp dụng vào mô hình ở Chương 2: First-Come, First-Served Batching (B0), Greedy Batching (B2), Large Neighborhood Search (LNS), Adaptive Large Neighborhood Search (ALNS) và Variable Neighborhood Search (VNS). Cấu trúc nghiệm là danh sách chuyến của từng nhân viên trong (2.5); chất lượng được tính bằng (2.19). Các phương pháp khác nhau ở cách tạo và thay đổi nghiệm, nhưng dùng cùng các đại lượng quãng đường, makespan và độ trễ.

Quy trình bắt đầu bằng kiểm tra đầu vào, tính khoảng cách trên đồ thị và xây dựng B0 để cố định thang chuẩn hóa. Nếu chọn B0, chương trình sử dụng nghiệm này làm đầu ra. Các phương pháp còn lại tạo B2; riêng LNS, ALNS và VNS tiếp tục cải thiện từ B2. Sau cùng, chương trình xuất tuyến, lịch, chỉ tiêu và kiểm chứng chúng trước khi trả kết quả.



Hình 3.1: Quy trình giải và kiểm chứng của năm phương pháp.

Ngân sách tìm kiếm của LNS, ALNS và VNS được tính từ trước khi tạo B2, nên bao gồm khởi tạo và tìm kiếm. Tiền xử lý, tạo mốc B0 và kiểm chứng đầu ra được đo riêng. Chương trình dừng theo ngân sách thời gian hoặc số vòng tối đa; các điểm kiểm tra thời gian không ngắt tức thời một phép dựng tuyến hoặc đánh giá đang thực hiện. Vì vậy, thời gian chạy tổng không được đồng nhất với ngân sách tìm kiếm, cũng không phải makespan của kế hoạch.

3.2 Định tuyến, lập lịch và đánh giá nghiệm

3.2.1 Xây dựng tuyến bằng Nearest Neighbor

Với tập vị trí $U(B)$ của một chuyến, Nearest Neighbor bắt đầu tại depot. Tại mỗi bước, thuật toán chọn vị trí chưa phục vụ gần nhất theo khoảng cách đường đi ngắn nhất δ; mã vị trí dùng để phá hòa. Sau khi phục vụ các vị trí yêu cầu ngoài depot, tuyến quay về depot. Dãy điểm dừng được mở rộng thành đường đi trên đồ thị như đã mô tả ở mục 2.2.

Quy tắc chọn tại điểm u có thể viết

$$u^* = \arg\min_{z \in U_{\text{remaining}}} (\delta(u, z), \text{id}(z)). \tag{3.1}$$

Đây là lựa chọn tham lam từng bước, không bảo đảm tuyến ngắn nhất trên toàn bộ tập vị trí. Với r vị trí ngoài depot và tra cứu khoảng cách đã có, việc chọn thứ tự điểm dừng cần $O(r^2)$ phép xét. Chi phí mở rộng đường đi vật lý được tính thêm theo độ dài dãy đỉnh xuất ra. B0 dùng trực tiếp quy tắc này; bốn phương pháp còn lại kết hợp với 2-Opt.

3.2.2 Cải thiện tuyến bằng 2-Opt

2-Opt xét hai cạnh (a, b) và (c, d) trong tuyến. Trên đồ thị vô hướng, đảo đoạn giữa b và c thay hai cạnh này bằng (a, c) và (b, d). Độ thay đổi quãng đường là

$$\Delta_D = \delta(a, c) + \delta(b, d) - \delta(a, b) - \delta(c, d). \tag{3.2}$$

Trong cài đặt, một biến đổi được nhận nếu $\Delta_D < -10^{-9}$. Thuật toán nhận cặp cải thiện đầu tiên theo thứ tự vòng lặp, đảo đoạn rồi bắt đầu lượt quét mới. Quá trình kết thúc khi một lượt quét không tìm thấy cải thiện; depot được giữ ở hai đầu tuyến.

Nếu ρ là số lần đảo đoạn được nhận, chi phí xét cặp là $O((\rho + 1)r^2)$, bao gồm lượt cuối không cải thiện. Đảo một đoạn còn cần $O(r)$ thao tác. Không xem ρ là hằng số hoặc mặc định số lượt cải thiện có cận đa thức theo r. Điều kiện dừng chỉ cho biết không còn cải thiện bằng phép đổi đang xét trong dung sai, không chứng minh tuyến tối ưu toàn cục.

3.2.3 Phân công nhân viên và xây dựng lịch thực hiện

Sau bước gom đơn cơ sở, các chuyến được sắp theo hạn nhỏ nhất của các đơn thành viên. Khi bằng nhau, chương trình giữ thứ tự chỉ số chuyến đã tạo. Lần lượt từng chuyến được giao cho nhân viên có thời điểm rảnh sớm nhất; mã nhân viên dùng để phá hòa. Thời điểm rảnh được cập nhật bằng thời lượng chuyến trong (2.10).

Nếu có b_{mtrip} chuyến, xác định hạn nhỏ nhất cần xét tổng cộng n đơn; sắp chuyến cần $O(b_{mtrip} \log b_{mtrip})$, còn lựa chọn nhân viên bằng quét danh sách cần $O(b_{mtrip} m)$. Chi phí tính thời lượng và tuyến được cộng riêng. Quy tắc gom theo thứ tự của B0 vì vậy không đồng nghĩa với việc mọi chuyến được thực hiện theo thứ tự đơn đến.

Trong tìm kiếm, phân công và thứ tự chuyến là một phần của nghiệm, có thể được thay đổi trực tiếp. Chương trình không tự sắp lại tất cả chuyến theo hạn sau mỗi biến đổi, vì thao tác đó có thể xóa quyết định lập lịch vừa tạo. Lịch được tính lại theo từng danh sách chuyến bằng (2.12).

3.2.4 Tính giá trị mục tiêu và kiểm tra ràng buộc

Một ứng viên hoàn chỉnh phải phục vụ đủ đơn, không trùng đơn và đúng sức chứa trước khi được xem là nghiệm đầu ra. Bộ đánh giá dựng tuyến theo chế độ đang dùng, tính thời lượng, lịch và các thành phần trong F. Khi một chuyến thay đổi, thời điểm của các chuyến phía sau cùng nhân viên cũng có thể thay đổi; không chỉ tính chi phí riêng của đơn vừa di chuyển.

Trong phá hủy–tái thiết, bộ đánh giá có thể nhận trạng thái tạm thiếu đơn để xếp hạng hành động chèn. Khi đó, chỉ tiêu chỉ được tính trên các đơn đang có trong trạng thái, nhưng thang chuẩn hóa vẫn cố định từ B0 của đầu vào đầy đủ. Điểm của trạng thái tạm không được coi là chất lượng một nghiệm hoàn chỉnh hoặc cận dưới đã chứng minh của bài toán.

Tuyến, thông tin chuyến và các tiền tố lịch được lưu trong bộ nhớ đệm để tái sử dụng khi cấu trúc tương ứng không đổi. Sau khi chọn đầu ra, bộ kiểm chứng đối chiếu đường đi, tải, thời gian và chỉ tiêu từ kết quả xuất ra. Cách tổ chức này gắn giá trị mục tiêu với kế hoạch thực sự được trình bày, thay vì chỉ tin vào một điểm số trung gian.

3.3 First-Come, First-Served Batching (B0)

3.3.1 Ý tưởng và nguyên lý

B0 tạo nghiệm cơ sở bằng cách duyệt các đơn theo thứ tự đầu vào. Chương trình sắp theo trường thứ tự, rồi theo mã đơn nếu bằng nhau. Mỗi đơn được thêm vào chuyến hiện tại nếu vừa sức chứa; nếu không, chuyến hiện tại được đóng và một chuyến mới được mở. Đầu vào bảo đảm từng đơn riêng lẻ vừa xe, nên bước này có thể phục vụ đủ đơn mà không cần tách đơn.

B0 không lựa chọn đơn theo độ gần vị trí hoặc mức giảm mục tiêu. Tuyến mỗi chuyến dùng Nearest Neighbor, còn phân công chuyến dùng quy tắc ở mục 3.2.3. Vai trò của B0 là cung cấp một kế hoạch dễ giải thích và mốc chuẩn hóa nhất quán, không

phải một cận tối ưu.

3.3.2 Các bước thực hiện và giả mã

Bước	Thao tác
	Đầu vào: tập đơn, tải đơn, sức chứa, đồ thị và thông số vận hành. Đầu ra: nghiệm B0.
1	Sắp đơn theo thứ tự đầu vào; phá hòa bằng mã đơn.
2	Khởi tạo danh sách chuyến rỗng, chuyến hiện tại $B = \varnothing$ và tải $w = 0$.
3	Với mỗi đơn i, nếu $B \neq \varnothing$ và $w + w_i > Q$ thì lưu B, đặt $B = \varnothing$ và $w = 0$.
4	Thêm i vào B và cập nhật tải.
5	Sau khi duyệt hết đơn, lưu chuyến cuối nếu không rỗng.
6	Tạo tuyến bằng Nearest Neighbor; sắp chuyến theo hạn nhỏ nhất và giao cho nhân viên rảnh sớm nhất.
7	Tính lịch, chỉ tiêu, thang chuẩn hóa; kiểm chứng và trả nghiệm.

Bảng 3.1: Giả mã B0 theo cài đặt của đề tài.

Các phép kiểm tra sức chứa trong mã có dùng dung sai số thực; giả mã viết điều kiện toán học để làm rõ ý tưởng. Các bước giải mã và kiểm chứng được tách ra để không lặp toàn bộ công thức của Chương 2.

3.3.3 Ví dụ minh họa

Với ví dụ ở mục 2.7, thứ tự đầu vào là O_1, O_2, O_3, O_4 và xe chứa được hai đơn. B0 tạo hai chuyến $\{O_1, O_2\}$ và $\{O_3, O_4\}$. Hai chuyến cùng có hạn nhỏ nhất là 7 phút, nên được xét theo thứ tự tạo và giao cho hai nhân viên. Quãng đường lần lượt là 16 và 18 mét; thời lượng là 11 và 12 phút.

Theo Bảng 2.3, B0 có $D = 34$, $C_{\max} = 12$, $T = 9$ và $F \approx 0{,}729167$. Việc ghép đơn gần depot với đơn xa làm các đơn có hạn gần phải chờ chuyến quay về. Ví dụ minh họa hạn chế của quy tắc gom theo thứ tự trên đầu vào này, không khẳng định B0 kém hơn trên mọi bài.

3.3.4 Phân tích chi phí tính toán và đặc điểm thuật toán

Sắp đơn cần $O(n \log n)$, còn duyệt tải và tạo chuyến cần $O(n)$. Cần cộng chi phí lập lịch và dựng tuyến cho từng chuyến; do đó, không mô tả toàn bộ B0 chỉ bằng $O(n \log n)$. Chi phí tiền xử lý đồ thị được trình bày riêng ở mục 3.8.

B0 tất định với cùng đầu vào và quy tắc phá hòa. Nó dễ tái lập, tạo nghiệm khả thi theo sức chứa và ít quyết định tìm kiếm, nhưng không trực tiếp tối ưu quan hệ giữa vị trí, hạn và tải công việc. Giá trị của B0 nằm ở vai trò mốc so sánh và kiểm tra, không phải bảo đảm chất lượng gần tối ưu.

3.4 Greedy Batching (B2)

3.4.1 Ý tưởng và nguyên lý

B2 chọn đơn có hạn sớm nhất trong tập chưa phục vụ làm hạt giống; mã đơn dùng để phá hòa. Trong các đơn còn vừa sức chứa, thuật toán chọn đơn làm tăng ít nhất quãng đường Nearest Neighbor của chuyến. Quy tắc được lặp đến khi không còn đơn vừa tải, rồi mở chuyến mới.

Gọi B là chuyến đang tạo, tiêu chí thêm đơn là

$$\Delta_i^{\mathrm{NN}} = D_{\mathrm{NN}}(B \cup \{i\}) - D_{\mathrm{NN}}(B), \qquad i^* = \arg\min_{i:\ \text{vừa tải}} (\Delta_i^{\mathrm{NN}}, d_i, \mathrm{id}(i)). \tag{3.3}$$

Hạn chỉ quyết định hạt giống và phá hòa của bước chọn này. B2 chưa xét trực tiếp toàn bộ F khi ghép đơn. Sau khi gom, chương trình dùng Nearest Neighbor kết hợp 2-Opt để tạo tuyến cuối, rồi phân công và lập lịch như mục 3.2.3. Cần phân biệt tiêu chí gom dựa trên tuyến Nearest Neighbor với tuyến đã được cải thiện của nghiệm xuất ra.

3.4.2 Các bước thực hiện và giả mã

Bước	Thao tác
	Đầu vào: các dữ liệu của B0 và thang chuẩn hóa cố định. Đầu ra: nghiệm B2.
1	Khởi tạo tập đơn chưa phục vụ $R = O$ và danh sách chuyến rỗng.
2	Khi $R \neq \varnothing$, lấy đơn có $(d_i, \mathrm{id}(i))$ nhỏ nhất làm chuyến B và bỏ khỏi R.
3	Lập tập các đơn trong R có thể thêm vào B mà không vượt tải.
4	Nếu tập này rỗng, lưu B và quay lại bước 2.
5	Tính Δ_i^{NN} cho từng đơn khả thi; chọn theo (3.3).
6	Thêm đơn đã chọn vào B, bỏ khỏi R và quay lại bước 3.
7	Dựng tuyến cuối bằng Nearest Neighbor và 2-Opt; phân công, lập lịch, đánh giá và kiểm chứng.

Bảng 3.2: Giả mã B2 theo cài đặt của đề tài.

3.4.3 Ví dụ minh họa

Trong ví dụ bốn đơn, O_1 và O_3 cùng có hạn 7 phút; B2 chọn O_1 do mã nhỏ hơn. Chuyến đơn $\{O_1\}$ có quãng đường 4 mét. Thêm O_3 làm quãng đường thành 6 mét, tăng 2 mét; thêm O_2 làm thành 16 mét, tăng 12 mét; thêm O_4 làm thành 18 mét, tăng 14 mét. Cả ba đều vừa tải, nên B2 ghép O_3 vào chuyến đầu.

Xe đã đầy, nên O_2 và O_4 được gom ở chuyến sau. Phân công mỗi nhân viên một chuyến tạo phương án B2 trong Bảng 2.3, với $D = 24$, $C_{\max} = 12$, $T = 0$ và

$F \approx 0{,}568627$. Trên lối đi này, 2-Opt không làm giảm thêm quãng đường. Vì vậy, phần giảm đường đi trong riêng ví dụ đến từ cách gom; điều này không được suy rộng cho mọi so sánh B0–B2.

3.4.4 Phân tích chi phí tính toán và đặc điểm thuật toán

Mỗi lần thêm đơn có thể xét nhiều đơn còn lại. Với tối đa n lần lựa chọn và tối đa n ứng viên mỗi lần, số lượt tính phương án thêm có cận $O(n^2)$. Mỗi lượt vẫn cần xác định vị trí và quãng đường chuyến; sắp các đơn còn lại theo mã còn tạo chi phí riêng. Khi gọi $\mathcal{C}_{\text{cand}}$ là chi phí tính một ứng viên gom, phần kiến tạo có cận dạng $O(n^2(\log n + \mathcal{C}_{\text{cand}}))$, chưa gồm tuyến cuối và lập lịch.

B2 tất định và xét độ gần của nhóm đơn, nhưng tiêu chí tham lam cục bộ không trực tiếp tối thiểu hóa F. Một phép ghép tiết kiệm đường đi có thể làm tăng thời điểm hoàn thành hoặc trễ hạn. B2 do đó không được bảo đảm luôn tốt hơn B0. Vai trò tiếp theo của B2 là nghiệm khởi tạo khả thi để LNS, ALNS và VNS có một mốc xuất phát chung.

3.5 Large Neighborhood Search (LNS)

3.5.1 Ý tưởng và nguyên lý

LNS cải thiện một nghiệm bằng cách loại bỏ một phần các đơn rồi chèn chúng trở lại để tạo cấu trúc khác. Nguyên lý này kế thừa hướng tìm kiếm lân cận lớn của Shaw [3]; các thao tác và tiêu chí chèn dưới đây mô tả riêng cài đặt của đề tài. LNS bắt đầu từ B2, lưu nghiệm hiện tại X và nghiệm tốt nhất X^* riêng biệt. Các toán tử phá hủy và tái thiết được chọn với trọng số bằng nhau.

Cấu hình dùng trong thực nghiệm chính có bốn toán tử phá hủy. Random Removal lấy mẫu đơn ngẫu nhiên. Related Removal chọn một đơn hạt giống i_0 rồi xếp các đơn theo mức liên quan

$$r(i_0, i) = \frac{\min_{u \in U(\{i_0\}), z \in U(\{i\})} \delta(u, z)}{D_{\text{ref}}} + \frac{|d_{i_0} - d_i|}{C_{\text{ref}}}, \tag{3.4}$$

phá hòa bằng mã đơn. Late Removal ưu tiên độ trễ giảm dần, rồi hạn và mã đơn. Batch Removal chọn một chuyến ngẫu nhiên và lấy toàn bộ đơn của chuyến đó ra. Các chuyến rỗng sau phá hủy được loại khỏi danh sách.

Với ba toán tử đầu, số đơn rút là

$$q = \min\{n, q_{\max}, \max(1, \lceil \eta n \rceil)\}. \tag{3.5}$$

Trong cấu hình chính, $\eta = 0{,}15$ và $q_{\max} = 6$. Batch Removal là ngoại lệ: nó bỏ cả chuyến, nên số đơn rút thực tế có thể lớn hơn $q_{\max}$. Cần dùng số đơn thực sự bị rút khi phân tích chi phí tái thiết.

Một hành động tái thiết có thể chèn đơn vào chuyến còn vừa tải, hoặc tạo chuyến mới tại một vị trí trong lịch của một nhân viên. Mỗi hành động được đánh giá bằng mục tiêu trên trạng thái tạm. Nếu số hành động vượt giới hạn ứng viên, cài đặt chọn ngẫu nhiên một hành động tạo chuyến mới bắt buộc và lấy mẫu các hành động còn lại. Cách này duy trì khả năng chèn lại một đơn riêng lẻ vừa xe, nhưng không bảo đảm xét hành động tốt nhất trong toàn bộ không gian. Phần chính dùng lấy mẫu uniform; giới hạn 24 áp dụng cho từng đơn đang xét, không phải cho toàn bộ lần tái thiết.

Hai quy tắc tái thiết dùng chung các hành động trên. Greedy Repair chọn đơn có điểm chèn tốt nhất nhỏ nhất. Regret-2 Repair ưu tiên đơn có chênh lệch lớn nhất giữa hai lựa chọn tốt nhất:

$$R_i = c_{i,2} - c_{i,1}, \qquad c_{i,1} \le c_{i,2}. \tag{3.6}$$

Sau khi chọn đơn, cả hai quy tắc thực hiện hành động tốt nhất của đơn đó, cập nhật trạng thái và đánh giá lại các đơn còn chờ. Nếu chỉ có một hành động, cài đặt đặt regret bằng vô cùng để ưu tiên đơn này. Các giá trị regret dựa trên danh sách ứng viên đã giữ, không nhất thiết dựa trên tất cả vị trí chèn có thể có.

Mỗi năm vòng hoàn chỉnh trong cấu hình chính, ứng viên còn được cải thiện cục bộ với tối đa tám lần thử. Mỗi lần lấy mẫu chuyển/đổi đơn giữa hai chuyến hoặc đổi/chuyển một chuyến trong lịch. Chỉ biến đổi cải thiện F mới được nhận trong bước cục bộ. Đây là bước có giới hạn số mẫu, không phải duyệt hết lân cận.

Ứng viên hoàn chỉnh được xét chấp nhận theo cơ chế Simulated Annealing [6]. Đặt $\Delta = F(Y) - F(X)$ và gọi θ là nhiệt độ tìm kiếm, khác với tổng độ trễ T ở Chương 2:

$$P_{\text{accept}}(Y \mid X) = \begin{cases} 1, & \Delta \le 0, \\ \exp\!\left(-\Delta / \max(\theta, 10^{-12})\right), & \Delta > 0. \end{cases} \tag{3.7}$$

Nhiệt độ giảm sau mỗi vòng hoàn chỉnh theo $\theta \leftarrow \gamma\theta$, với $0 < \gamma < 1$. Việc cho phép nhận ứng viên xấu giúp thay đổi đường tìm kiếm, nhưng không bảo đảm thoát mọi cực trị hoặc tìm được tối ưu trong ngân sách hữu hạn.

3.5.2 Các bước thực hiện và giả mã

Bước	Thao tác
	Đầu vào: B2, thang chuẩn hóa, seed, toán tử và điều kiện dừng. Đầu ra: nghiệm tốt nhất đã lưu.
1	Đặt $X = X^* = B2$; tính điểm và khởi tạo nhiệt độ.
2	Khi còn số vòng và ngân sách, chọn một toán tử phá hủy và một toán tử tái thiết với trọng số bằng nhau.
3	Rút các đơn đã chọn khỏi X, loại chuyến rỗng để tạo trạng thái tạm.
4	Đánh giá các hành động chèn; dùng Greedy Repair hoặc Regret-2 Repair đến khi chèn lại đủ đơn.
5	Nếu đến chu kỳ cải thiện cục bộ, thực hiện các lần thử đã giới hạn; chỉ nhận cải thiện.
6	Tính $F(Y)$ của ứng viên hoàn chỉnh. Nếu hết thời gian trong bước 3–5, bỏ lần đang thực hiện và chuyển đến bước 10.
7	Chấp nhận Y theo (3.7); nếu được nhận, đặt $X = Y$.
8	Nếu $F(Y) < F(X^*) - 10^{-12}$ thì lưu $X^* = Y$.
9	Giảm nhiệt độ; chuyển sang vòng tiếp theo. Trọng số toán tử vẫn bằng 1.
10	Kiểm chứng và trả X^*, không trả trạng thái tạm thiếu đơn.

Bảng 3.3: Giả mã LNS với lựa chọn toán tử đều.

Giả mã tách bước chấp nhận nghiệm hiện tại và bước ghi nhận nghiệm tốt nhất như cài đặt. Việc nhận một ứng viên xấu không xóa nghiệm tốt nhất đã có. Số vòng hoàn chỉnh có thể bằng 0 nếu ngân sách đã hết sau khởi tạo.

3.5.3 Ví dụ minh họa

Để minh họa bước tái thiết trên dữ liệu ở mục 2.7, lấy O_2, O_3 ra khỏi phương án B0. Trạng thái còn lại có $\{O_1\}$ ở nhân viên thứ nhất và $\{O_4\}$ ở nhân viên thứ hai. Đây là một trạng thái minh họa cho thao tác, không phải tuyên bố rằng LNS khởi tạo từ B0; trong lượt giải chính, LNS khởi tạo từ B2.

Với các hành động được xét đầy đủ trên bài nhỏ, hai điểm tốt nhất của mỗi đơn cần chèn là:

Đơn	Điểm tốt nhất $c_{i,1}$	Điểm thứ hai $c_{i,2}$	Regret
O_2	0,549020	0,666667	0,117647
O_3	0,540850	0,583742	0,042892

Bảng 3.4: Các lựa chọn chèn trong ví dụ tái thiết hai đơn.

Greedy Repair chèn O_3 trước vì điểm tốt nhất nhỏ hơn; Regret-2 Repair chèn O_2 trước vì regret lớn hơn. Sau khi cập nhật và chèn đơn còn lại, cả hai cùng tạo các

chuyến $\{O_1, O_3\}$ và $\{O_2, O_4\}$ với $F \approx 0{,}568627$. Ví dụ làm rõ khác biệt trong thứ tự quyết định, không chứng minh hai quy tắc luôn cho cùng nghiệm hoặc quy tắc nào tốt hơn.

Nếu một ứng viên có $\Delta = 0{,}02$ tại $\theta = 0{,}03$, xác suất nhận theo (3.7) là $\exp(-0{,}02/0{,}03) \approx 0{,}5134$. Đây là phép tính minh họa cơ chế chấp nhận, không phải xác suất thành công của LNS trên benchmark.

3.5.4 Phân tích chi phí tính toán và đặc điểm thuật toán

Nếu một lần tái thiết rút q đơn và xét tối đa A hành động cho mỗi đơn đang chờ, số lượt đánh giá hành động có cận

$$A \sum_{j=1}^{q} j = \frac{Aq(q+1)}{2}. \tag{3.8}$$

Trong cấu hình chính, có thể lấy $A = 24$; khi tắt giới hạn ứng viên, phải dùng số vị trí chèn thực tế. Mỗi lượt còn tốn chi phí sao chép nghiệm, giải mã tuyến, tính lịch và kiểm tra ràng buộc. Lập danh sách, tính regret và sắp lựa chọn cũng có chi phí, nên số vòng tìm kiếm không đủ để phản ánh toàn bộ thời gian tính toán.

LNS có thể thay đổi nhiều đơn cùng lúc và xét ảnh hưởng lên cả ba thành phần mục tiêu. Tuy nhiên, chi phí tái thiết tăng theo số đơn rút; lấy mẫu ứng viên có thể bỏ lỡ vị trí tốt và Simulated Annealing cần tham số nhiệt độ. Trong dung sai ghi nhận nghiệm tốt nhất, đầu ra không kém B2 theo cùng bộ đánh giá. Tính chất này không bảo đảm tốt hơn một phương pháp tìm kiếm khác hoặc đạt tối ưu toàn cục.

3.6 Adaptive Large Neighborhood Search (ALNS)

3.6.1 Ý tưởng và nguyên lý

ALNS dùng cùng toán tử phá hủy, tái thiết, cải thiện cục bộ và cơ chế chấp nhận như LNS, nhưng điều chỉnh xác suất chọn toán tử dựa trên kết quả đã quan sát. Hướng lựa chọn thích ứng tham khảo Ropke và Pisinger [4]; thang thưởng và cập nhật dưới đây là các quy tắc của cài đặt đang được đánh giá.

Gọi $\omega_h > 0$ là trọng số của toán tử h. Các toán tử phá hủy và tái thiết được chọn riêng trong từng nhóm $\mathcal{D}$ và $\mathcal{R}$:

$$p_h = \frac{\omega_h}{\sum_{g \in \mathcal{G}} \omega_g}, \qquad \mathcal{G} \in \{\mathcal{D}, \mathcal{R}\}. \tag{3.9}$$

Ban đầu mọi trọng số bằng 1. Sau một vòng hoàn chỉnh, cả toán tử phá hủy và tái thiết đã dùng nhận cùng một điểm thưởng theo kết quả của ứng viên:

Kết quả của vòng tìm kiếm	Điểm thưởng
Cải thiện nghiệm tốt nhất đã lưu	8
Được chấp nhận và cải thiện nghiệm hiện tại, nhưng chưa cải thiện tốt nhất	3
Được chấp nhận trong trường hợp còn lại	1
Không được chấp nhận	0

Bảng 3.5: Điểm thưởng cho toán tử trong cài đặt ALNS.

Các điều kiện cải thiện dùng dung sai 10^{-12}. Thưởng dựa trên ứng viên sau cả tái thiết và bước cục bộ nếu có; vì vậy nó là tín hiệu hiệu quả của một vòng kết hợp, không phải phép đo tác động độc lập của từng toán tử.

Sau một phân đoạn, gọi N_h là số lần toán tử được dùng và R_h là tổng thưởng trong đoạn. Với $N_h > 0$, trọng số được cập nhật bởi

$$\omega_h \leftarrow \max\left\{0{,}05,\, (1-\lambda)\omega_h + \lambda\frac{R_h}{N_h}\right\}. \tag{3.10}$$

Toán tử chưa được dùng giữ nguyên trọng số; sau cập nhật, số lần dùng và tổng thưởng của đoạn được đặt lại bằng 0. Sàn $0{,}05$ giữ xác suất dương, nhưng không bảo đảm mọi toán tử được chọn trong một lượt chạy hữu hạn. Chương trình cũng cập nhật phần thống kê còn lại khi kết thúc lượt giải.

3.6.2 Các bước thực hiện và giả mã

Bước	Thao tác
	Đầu vào: dữ liệu của LNS, độ dài phân đoạn và hệ số phản ứng. Đầu ra: nghiệm tốt nhất đã lưu.
1	Đặt $X = X^* = B2$; khởi tạo nhiệt độ và mọi trọng số toán tử bằng 1.
2	Đặt thống kê số lần dùng và tổng thưởng của từng toán tử bằng 0.
3	Khi còn ngân sách và số vòng, chọn phá hủy và tái thiết theo (3.9).
4	Phá hủy, tái thiết và cải thiện cục bộ định kỳ như LNS.
5	Nếu hết thời gian trước khi hoàn tất ứng viên, bỏ vòng đang thực hiện và đến bước 12.
6	Tính $F(Y)$, đặt $\Delta = F(Y) - F(X)$; xét chấp nhận theo (3.7).
7	Nếu được nhận, cập nhật X và thưởng 3 khi cải thiện hiện tại, hoặc 1 trong trường hợp còn lại. Nếu bị từ chối, thưởng 0.
8	Nếu cải thiện X^*, lưu nghiệm tốt nhất và thay thưởng bằng 8.
9	Cộng thưởng và tăng số lần dùng cho cả hai toán tử đã chọn.
10	Khi hết phân đoạn, cập nhật trọng số theo (3.10) và đặt lại thống kê đoạn.
11	Giảm nhiệt độ; tiếp tục vòng sau.
12	Cập nhật thống kê đoạn còn lại; kiểm chứng và trả X^*.

Bảng 3.6: Giả mã ALNS với lựa chọn toán tử thích ứng.

Trong cấu hình của đợt benchmark chính, phân đoạn có 10 vòng và $\lambda = 0{,}25$. Cần phân biệt cấu hình thực nghiệm này với giá trị mặc định của lớp cấu hình. Nếu lượt chạy kết thúc trước một lần cập nhật được dùng cho các vòng tiếp theo, tác động của lựa chọn thích ứng có thể chưa xuất hiện trong quá trình tìm kiếm.

3.6.3 Ví dụ minh họa

Giả sử một toán tử có trọng số 1, được dùng bốn lần trong đoạn và nhận tổng thưởng 12. Với $\lambda = 0{,}25$, cập nhật cho trọng số mới

$$\omega'_h = \max\{0{,}05,\, 0{,}75 \times 1 + 0{,}25 \times 12/4\} = 1{,}5. \tag{3.11}$$

Nếu ba toán tử phá hủy còn lại vẫn có trọng số 1, xác suất chọn toán tử này trong nhóm tăng từ $1/4$ lên $1{,}5/4{,}5 = 1/3$. Ví dụ mô tả tác động của điểm thưởng lên xác suất, không kết luận toán tử đó luôn tạo nghiệm tốt.

Bước tái thiết của ALNS vẫn có thể dùng các lựa chọn trong Bảng 3.4. Điểm khác LNS nằm ở cách chọn toán tử qua các đoạn, không nằm ở việc thay đổi hàm mục tiêu hoặc tự động chọn hành động chèn tốt nhất trong toàn bộ không gian. Khi các toán tử tạo cùng kết quả trên một bài nhỏ, hai phương pháp có thể trả cùng điểm mà chưa cho phép kết luận thích ứng vô ích trên mọi dữ liệu.

3.6.4 Phân tích chi phí tính toán và đặc điểm thuật toán

Chi phí tạo ứng viên của ALNS có cùng cấu trúc với LNS. Phần bổ sung là lấy mẫu theo trọng số, ghi thưởng và cập nhật các toán tử theo phân đoạn. Với số toán tử nhỏ và cố định, phần này thường nhỏ hơn chi phí dựng tuyến và đánh giá nhiều hành động chèn, nhưng mức chênh lệch thời gian cần được đo thay vì mặc định bằng không.

ALNS khai thác thông tin của các vòng đã chạy để thay đổi tần suất sử dụng toán tử. Hiệu quả phụ thuộc vào số vòng có thể hoàn thành, độ dài phân đoạn, hệ số phản ứng và đặc điểm đầu vào. LNS là đối chứng cho phần lựa chọn thích ứng khi giữ chung các thành phần khác. Trong ngân sách thời gian bằng nhau, số vòng và chuỗi ngẫu nhiên của hai phương pháp vẫn có thể khác; việc đánh giá cơ chế cần giữ phạm vi kết luận theo thiết kế thực nghiệm.

Như LNS, ALNS trả nghiệm tốt nhất đã ghi nhận, nên dãy điểm tốt nhất không tăng và đầu ra không kém nghiệm khởi tạo trong dung sai. Nghiệm hiện tại có thể xấu đi do cơ chế chấp nhận. Hai tính chất này cần được phân biệt khi đọc đường hội tụ; chúng không tạo bảo đảm tối ưu toàn cục hoặc ưu thế của ALNS so với VNS.

3.7 Variable Neighborhood Search (VNS)

3.7.1 Ý tưởng và nguyên lý

VNS thay đổi cấu trúc lân cận để tìm kiếm từ nhiều cách biến đổi nghiệm, theo nguyên lý của Mladenović và Hansen [5]. Cài đặt của đề tài bắt đầu từ B2, thực hiện rung lắc rồi cải thiện bằng một số mẫu lân cận có giới hạn. Nghiệm sau bước này chỉ thay thế nghiệm tốt nhất khi có cải thiện; VNS không dùng cơ chế Simulated Annealing của LNS và ALNS.

Ba nhóm lân cận được dùng là Order Neighborhood, Schedule Neighborhood và Batch Neighborhood. Nhóm thứ nhất chuyển một đơn giữa hai chuyến hoặc đổi hai đơn thuộc hai chuyến, với tải sau thay đổi không vượt sức chứa. Nhóm thứ hai đổi hai chuyến hoặc lấy một chuyến ra rồi chèn vào vị trí khác, có thể chuyển sang nhân viên khác. Nhóm thứ ba tách một phần đơn thành chuyến mới, hoặc gộp hai chuyến nếu tổng tải cho phép. Chuyến rỗng sau biến đổi được loại bỏ; thao tác không khả thi có thể trả lại nghiệm ban đầu.

Gọi $\kappa \in \{1, \ldots, 6\}$ là mức rung lắc. Nhóm được chọn theo

$$g = 1 + ((\kappa - 1) \bmod 3), \tag{3.12}$$

và một lần rung lắc gọi nhóm đó κ lần liên tiếp. Mức lớn hơn vì vậy tăng số thao tác được thử, không bảo đảm mỗi thao tác tạo ra một thay đổi khác biệt.

Mức rung lắc	Nhóm lân cận	Số lần gọi
$1, 4$	Chuyển hoặc đổi đơn giữa các chuyến	$1, 4$
$2, 5$	Đổi hoặc chuyển vị trí chuyến trong lịch	$2, 5$
$3, 6$	Tách hoặc gộp chuyến	$3, 6$

Bảng 3.7: Quan hệ giữa mức rung lắc và nhóm lân cận của VNS.

Sau rung lắc, bước cải thiện bắt đầu từ nhóm chuyển/đổi đơn. Nếu một mẫu làm giảm F quá dung sai 10^{-12}, ứng viên được cập nhật và việc chọn nhóm quay về nhóm đầu. Nếu không, chuyển tuần hoàn sang nhóm tiếp theo. Tổng số lần thử bị giới hạn bởi tham số cải thiện cục bộ; cấu hình chính dùng tám lần. Cách này là cải thiện trên mẫu, không phải duyệt hết lân cận hoặc xác nhận một cực trị cục bộ đầy đủ.

Nếu ứng viên cuối tốt hơn nghiệm tốt nhất, chương trình nhận ứng viên và đặt $\kappa = 1$. Nếu không, giữ nghiệm tốt nhất và đặt $\kappa \leftarrow 1 + (\kappa \bmod 6)$. Rung lắc có thể tạo ứng viên tạm xấu hơn, nhưng ứng viên đó không được trả như nghiệm tốt nhất chỉ vì đã được tạo ra.

3.7.2 Các bước thực hiện và giả mã

Bước	Thao tác
	Đầu vào: B2, seed, ba nhóm lân cận và điều kiện dừng. Đầu ra: nghiệm tốt nhất đã lưu.
1	Đặt $X^* = B2$, tính $F(X^*)$ và khởi tạo $\kappa = 1$.
2	Khi còn ngân sách và số vòng, đặt ứng viên $Y = X^*$.
3	Chọn nhóm g theo (3.12); áp dụng nhóm này κ lần để rung lắc Y.
4	Tính $F(Y)$ và đặt nhóm cải thiện $j = 1$.
5	Trong số lần thử cho phép, lấy một mẫu Z từ nhóm j của Y và tính $F(Z)$.
6	Nếu $F(Z) < F(Y) - 10^{-12}$, đặt $Y = Z$ và $j = 1$; nếu không, đặt $j = 1 + (j \bmod 3)$.
7	Nếu $F(Y) < F(X^*) - 10^{-12}$, lưu $X^* = Y$ và đặt $\kappa = 1$; nếu không, tăng mức tuần hoàn.
8	Nếu phát hiện hết thời gian trong lần đang thực hiện, dừng và giữ nghiệm tốt nhất đã ghi nhận.
9	Kiểm chứng và trả X^*.

Bảng 3.8: Giả mã VNS với cải thiện lân cận trên số mẫu giới hạn.

3.7.3 Ví dụ minh họa

Phương án X ở mục 2.7 giao hai chuyến $\{O_1, O_3\}$ và $\{O_2, O_4\}$ cho cùng một nhân viên. Một thao tác thuộc Schedule Neighborhood chuyển chuyến thứ hai sang nhân viên đang không có chuyến, giữ nguyên tập đơn và tuyến của hai chuyến. Khi đó, hai chuyến cùng bắt đầu tại 0, makespan giảm từ 18 xuống 12 phút và tổng độ trễ giảm từ 12 xuống 0; quãng đường vẫn là 24 mét.

Điểm mục tiêu giảm từ $0{,}818627$ xuống $0{,}568627$, chênh lệch $0{,}25$ với cùng thang chuẩn hóa. Đây là minh họa một thao tác lập lịch được hỗ trợ trong cài đặt, không phải mô tả một lượt VNS khởi tạo từ X. Lượt giải của VNS vẫn khởi tạo từ B2; ví dụ cho thấy vì sao lân cận phân công chuyến có thể hữu ích khi kế hoạch hiện tại mất cân bằng.

Trong ví dụ khác của cùng tập đơn, một bước chuyển/đổi đơn có thể thay đổi nhóm vị trí cần ghé; tách/gộp chuyến có thể thay đổi cả thời gian chuẩn bị và lịch. Các biến đổi này cần được so sánh bằng toàn bộ F, không chỉ bằng khoảng cách của một đơn hay một chuyến.

3.7.4 Phân tích chi phí tính toán và đặc điểm thuật toán

Một vòng VNS gọi tối đa sáu thao tác rung lắc và ℓ thao tác cải thiện, trong đó $\ell = 8$ ở cấu hình chính. Sau rung lắc có một lượt tính điểm; mỗi mẫu cải thiện có thêm một lượt. Gọi $\mathcal{C}_N$ là chi phí tạo một biến đổi và $\mathcal{C}_F$ là chi phí đánh giá một nghiệm, phần tìm kiếm qua I vòng có cận dạng

$$O\big(I\,[(6+\ell)\mathcal{C}_N + (1+\ell)\mathcal{C}_F]\big)\,. \tag{3.13}$$

Chi phí khởi tạo, xuất và kiểm chứng được cộng riêng. $\mathcal{C}_N$ gồm sao chép cấu trúc, kiểm tra tải và chuẩn hóa danh sách đơn; $\mathcal{C}_F$ vẫn phụ thuộc vào tuyến, lịch và bộ nhớ đệm. Khi dừng theo thời gian, I là số vòng thực sự hoàn thành, không phải luôn bằng giới hạn cấu hình. Nếu một vòng bị ngắt, chi phí đã thực hiện của vòng đó còn phải cộng vào thời gian chạy thực tế.

VNS tác động trực tiếp lên gom đơn và lịch, không cần tái chèn toàn bộ một tập đơn đã rút. Tuy nhiên, số mẫu hữu hạn có thể bỏ lỡ biến đổi tốt; một thao tác thất bại hoặc trả lại nghiệm cũ vẫn tiêu tốn thời gian. Nghiệm tốt nhất không bị thay bằng ứng viên xấu, nhưng việc tuần hoàn lân cận không tạo bảo đảm thoát mọi cực trị hoặc tìm tối ưu toàn cục.

3.8 So sánh cơ chế và phân tích chi phí tính toán

Các phương pháp được đối chiếu theo thành phần cài đặt, không chỉ theo tên gọi. Bảng 3.9 cho thấy B2 và ba phương pháp tìm kiếm dùng chung bước định tuyến cuối; LNS và ALNS giữ chung cơ chế tạo ứng viên, còn VNS thay đổi cấu trúc tìm kiếm.

Phương pháp	Tạo nghiệm hoặc ứng viên	Định tuyến	Chấp nhận trong tìm kiếm
B0	Gom theo thứ tự đầu vào	Nearest Neighbor	Không có pha tìm kiếm
B2	Gom tham lam theo hạn và tăng quãng đường	Nearest Neighbor và 2-Opt	Không có pha tìm kiếm
LNS	Phá hủy–tái thiết, chọn toán tử đều	Nearest Neighbor và 2-Opt	Simulated Annealing; lưu tốt nhất riêng
ALNS	Phá hủy–tái thiết, chọn toán tử thích ứng	Nearest Neighbor và 2-Opt	Như LNS; thêm cập nhật trọng số
VNS	Rung lắc và cải thiện trên mẫu lân cận	Nearest Neighbor và 2-Opt	Chỉ cập nhật tốt nhất khi cải thiện

Bảng 3.9: Đối chiếu các thành phần của năm phương pháp.

Gọi $N_V = |V|$, $N_E = |E|$ và h là số vị trí nguồn được tiền xử lý, gồm depot và các vị trí sản phẩm phân biệt. Với cách dùng Dijkstra trong chương trình, chi phí tiền xử lý khoảng cách có dạng $O(h(N_V + N_E) \log N_V)$, còn bảng khoảng cách và truy vết dùng bộ nhớ $O(hN_V)$ ngoài đồ thị. Phần này phục vụ chung các lần tính tuyến trong một lượt giải, không được coi là chi phí bằng không.

Để mô tả chi phí còn lại, gọi $\mathcal{C}_B(B)$ là chi phí tính thông tin một chuyến, gồm vị trí cần ghé, tải, tuyến và thời lượng; $\mathcal{C}_E$ là chi phí tạo và chấm một hành động chèn; $\mathcal{C}_F$ là chi phí chấm nghiệm. Các đại lượng này phụ thuộc kích thước chuyến, số điểm

lấy hàng, số lần cải thiện 2-Opt và trạng thái bộ nhớ đệm. Không mặc định mỗi lần đánh giá chỉ cần thời gian hằng số.

Thành phần	Chi phí hoặc số thao tác cần phân biệt
B0	Sắp đơn $O(n \log n)$; gom $O(n)$; lập lịch $O(b_{\text{trip}} \log b_{\text{trip}} + b_{\text{trip}} m)$; cộng thông tin tuyến/chuyến.
B2	Tối đa $O(n^2)$ lượt xét thêm đơn, mỗi lượt có chi phí ứng viên; cộng sắp các đơn còn lại, tuyến cuối và lập lịch.
Tái thiết LNS/ALNS	Với q đơn rút và tối đa A hành động mỗi đơn: không quá $Aq(q+1)/2$ lượt chấm hành động, ngoài lập và sắp lựa chọn.
Thích ứng ALNS	Ghi thưởng, lấy mẫu theo trọng số và cập nhật theo phân đoạn; không thay thế chi phí tái thiết.
VNS	Mỗi vòng có tối đa $6+\ell$ lần tạo biến đổi và $1+\ell$ lần chấm điểm; cộng khởi tạo và kiểm chứng.

Bảng 3.10: Các thành phần chi phí chính theo cài đặt.

Với LNS/ALNS, có thể cộng chi phí từng vòng theo

$$\begin{aligned} \mathcal{C}_{\text{search}} = \sum_{t=1}^{I} (&\mathcal{C}_{\text{destroy},t} + \mathcal{C}_{\text{repair},t} \\ &+ \mathcal{C}_{\text{local},t} + \mathcal{C}_{\text{control},t}) + \mathcal{C}_{\text{interrupted}}. \end{aligned} \tag{3.14}$$

Trong đó phần điều khiển bao gồm chấp nhận, lưu tốt nhất và thích ứng nếu có; $\mathcal{C}_{\text{interrupted}}$ là chi phí đã thực hiện của vòng bị ngắt, bằng 0 nếu không có vòng này. Số vòng hoàn chỉnh vì vậy không phản ánh mọi công việc đã thực hiện. Khi xét chi phí tổng, còn phải tính tiền xử lý, khởi tạo, xuất và kiểm chứng.

Số phần tử bộ nhớ đệm có giới hạn không đồng nghĩa với một mức dung lượng cố định tính bằng byte. Khóa và giá trị có kích thước khác nhau theo số đơn, số chuyến và độ dài đường đi. Việc cắt bỏ một mục đệm làm tăng khả năng phải tính lại, không thay đổi định nghĩa hàm mục tiêu. Tuy nhiên, trong ngân sách thời gian cố định, tốc độ khác nhau có thể làm thay đổi số vòng hoàn thành và nghiệm cuối. Trong thực nghiệm, thời gian chạy, số lần đánh giá và số vòng được dùng để bổ sung cho phân tích cấu trúc này.

Ba câu hỏi so sánh chính được nối với mục tiêu ở Chương 1: B0–B2 đánh giá hai cách kiến tạo có cả khác biệt định tuyến; so sánh với B2 đánh giá lợi ích của tiếp tục tìm kiếm; LNS–ALNS đối chiếu lựa chọn toán tử, còn VNS đối chiếu một chiến lược biến đổi nghiệm khác. Không dùng độ phức tạp hoặc tên phương pháp để suy ra trước thứ hạng chất lượng nghiệm.

4 THỰC NGHIỆM VÀ PHÂN TÍCH KẾT QUẢ

4.1 Thiết kế thực nghiệm

4.1.1 Câu hỏi và mục tiêu đánh giá

4.1.2 Các bộ dữ liệu và quy mô bài toán

4.1.3 Cấu hình thuật toán, ngân sách và số lần chạy

4.1.4 Môi trường thực nghiệm và phương pháp tổng hợp kết quả

4.2 Kết quả tổng hợp của năm thuật toán

4.2.1 Chất lượng nghiệm trên dữ liệu mô phỏng

4.2.2 Chất lượng nghiệm trên dữ liệu tham chiếu

4.2.3 Các chỉ tiêu vận hành và thời gian tính toán

4.3 Phân tích các trường hợp nổi bật

4.4 Phân tích quá trình hội tụ và độ ổn định

4.5 Khả năng mở rộng theo quy mô bài toán

4.6 Ảnh hưởng của ngân sách tìm kiếm

4.7 Ảnh hưởng của trọng số và đánh đổi giữa các mục tiêu

4.8 Giao diện minh họa và khả năng tái lập kết quả

4.9 Thảo luận tổng hợp và giới hạn thực nghiệm

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được theo mục tiêu đề tài

5.2 Nhận xét về điều kiện phù hợp của từng thuật toán

5.3 Hạn chế và hướng phát triển

TÀI LIỆU THAM KHẢO

[1] R. de Koster, T. Le-Duc, and K. J. Roodbergen, "Design and control of warehouse order picking: A literature review," *European Journal of Operational Research*, vol. 182, no. 2, pp. 481–501, 2007.

[2] O. Briant, H. Cambazard, N. Catusse, D. Cattaruzza, A.-L. Ladier, and M. Ogier, "Lower and upper bounds for the joint batching, routing and sequencing problem," *arXiv preprint arXiv:2303.17834v1*, 2023.

[3] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming – CP98*, LNCS, vol. 1520, pp. 417–431, Springer, 1998.

[4] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[5] N. Mladenović and P. Hansen, "Variable neighborhood search," *Computers & Operations Research*, vol. 24, no. 11, pp. 1097–1100, 1997.

[6] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[7] N. Gademann and S. van de Velde, "Order batching to minimize total travel time in a parallel-aisle warehouse," *IIE Transactions*, vol. 37, no. 1, pp. 63–75, 2005.

[8] H. D. Ratliff and A. S. Rosenthal, "Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem," *Operations Research*, vol. 31, no. 3, pp. 507–521, 1983.

[9] H. Cambazard, "Joint batching, routing and sequencing problem," *Trang dữ liệu nghiên cứu G-SCOP*, truy cập ngày 03/10/2026.

[10] TS. Trần Đình Tân, "Thuật toán ứng dụng," Trường Công nghệ Thông tin Phenikaa.

[11] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[12] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner, "Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents," *arXiv preprint arXiv:2609.00065*, 2026.

[13] T. Curtois, D. Landa-Silva, Y. Qu, and W. Laesanklang, "Large neighbourhood search with adaptive guided ejection search for the pickup and delivery problem with time windows," *EURO Journal on Transportation and Logistics*, vol. 7, pp. 151–192, 2018.

[14] NIST/SEMATECH, "Randomized block designs," *e-Handbook of Statistical Methods*, truy cập ngày 10/10/2026.